Exp No: 15

EMBEDDED C PROGRAM TO CONVERT THE HEXADECIMAL DATA 0XCFH TO DECIMAL AND DISPLAY THE DIGITS ON PORTS P0, P1 AND P2

AIM:

To write an Embedded C program to convert Hexadecimal numbers into Decimal and display the digits on ports using Keil software.

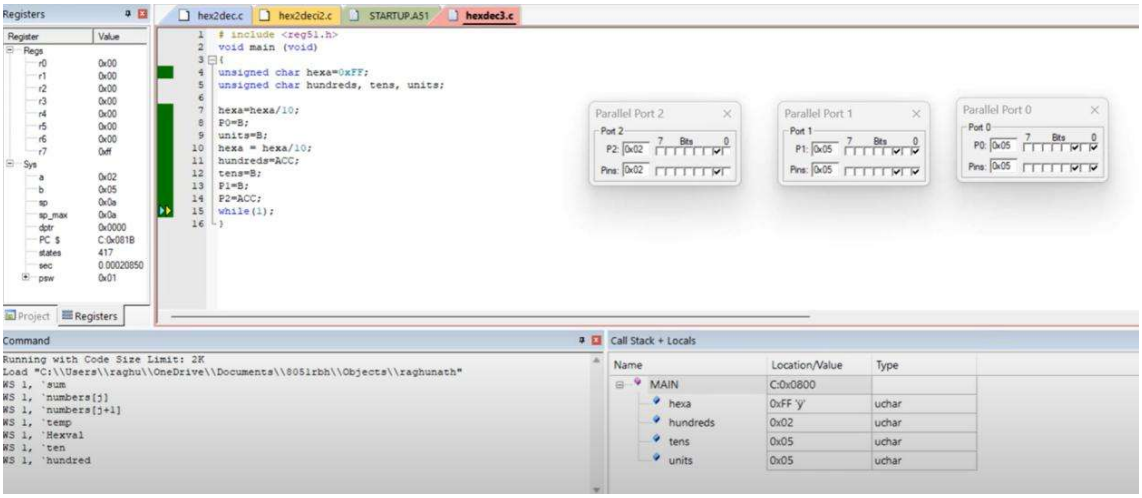
SOFTWARE REQUIRED:

- Keil software

PROGRAM:

```
#include <reg51.h>
void main(void)
{
   unsigned char hexa = 0x08;
   unsigned char hundreds, tens, units;

   ACC = hexa;
   B = 10;
   ACC = ACC / B;
   units = B;

   B = 10;
   ACC = ACC / B;
   tens = B;
   hundreds = ACC;

   P0 = units;
   P1 = tens;
   P2 = hundreds;

   while(1);
}
```

OUTPUT:



RESULT:

Thus the program has been successfully verified and executed.

Exp No:16

SERIAL INTERRUPT USING UART (KEYBOARD INPUT) IN KEIL

AIM:

To write and simulate an Embedded C program using **serial interrupt** in 8051 microcontroller such that while a message is continuously transmitted, any key pressed from the keyboard generates an interrupt and is displayed through UART.

APPARATUS / SOFTWARE REQUIRED:

- PC with Windows OS
- Keil µVision IDE
- 8051 Simulator (Keil built-in)
- Serial Window (UART #1)

THEORY:

The 8051 microcontroller provides a **serial interrupt** that occurs when:

- **RI (Receive Interrupt flag)** is set after receiving a character, or
- **TI (Transmit Interrupt flag)** is set after transmission.

In this experiment:

- UART is configured in **Mode 1 (8-bit UART)**
- Timer1 is used to generate baud rate (9600)
- Serial interrupt is enabled using ES bit
- When a key is typed in the **Keil Serial Window**, the received character sets RI
- The **Serial Interrupt Service Routine (ISR)** is executed immediately

This demonstrates **interrupt-driven serial communication**, which is more efficient than polling.

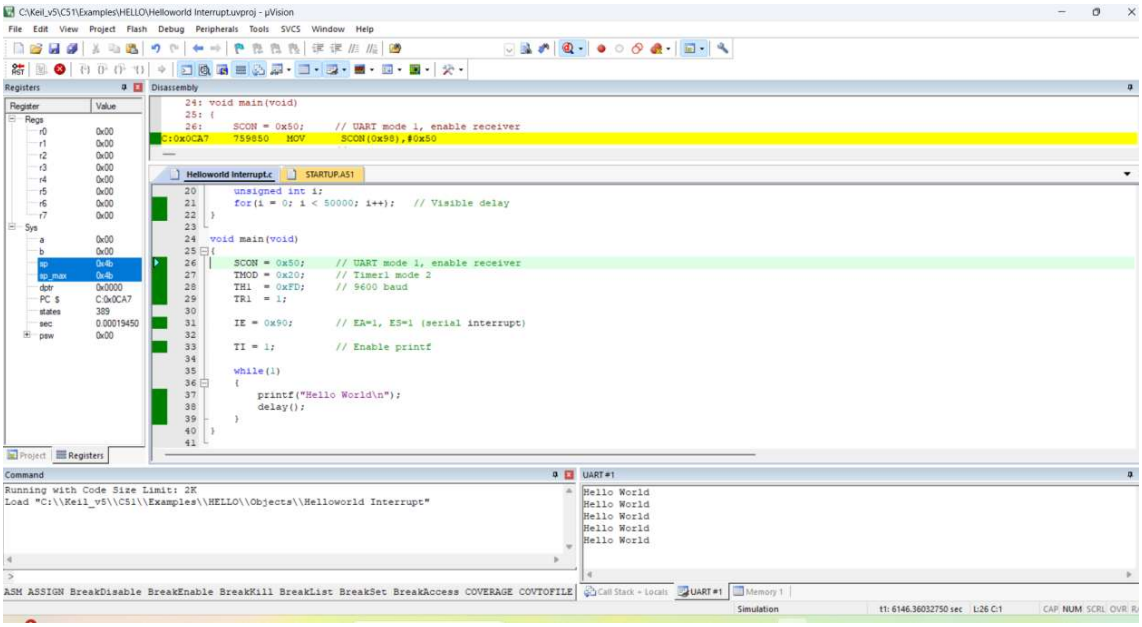
PROGRAM:

```
#include <reg51.h>
#include <stdio.h>


void serial_ISR(void) interrupt 4
{
   char ch;
   if (RI)
   {
```

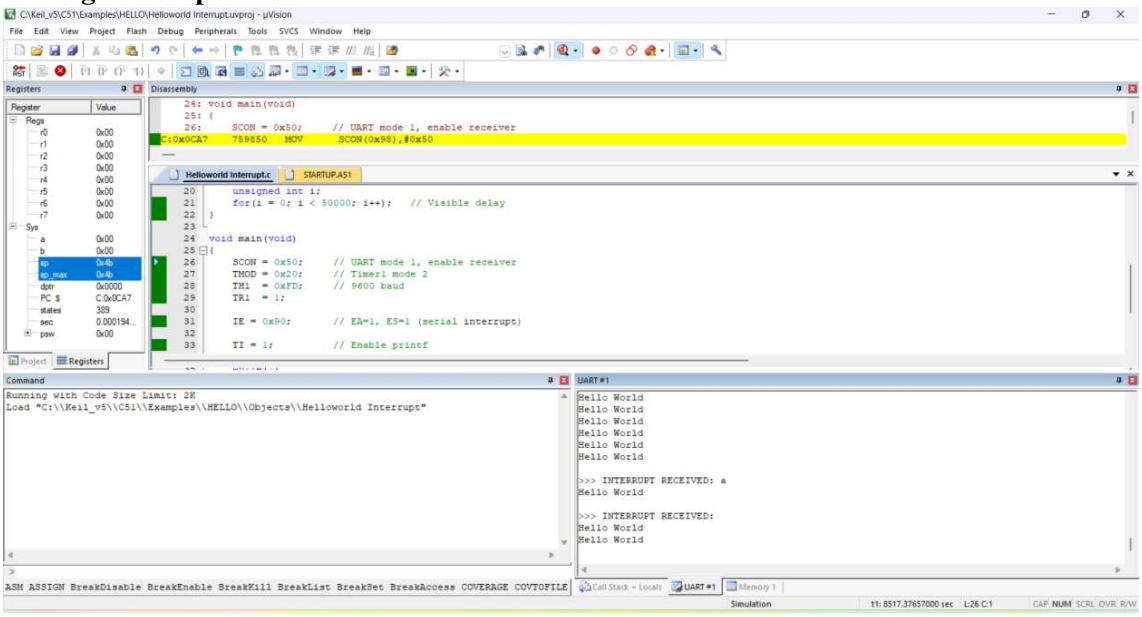
```
        RI = 0;          // Clear receive interrupt flag
        ch = SBUF;       // Read received character

        printf("\n>>> INTERRUPT RECEIVED: ");
        printf("%c\n", ch);
    }
}

void delay(void)
{
    unsigned int i;
    for(i = 0; i < 50000; i++);   // Delay for visibility
}

void main(void)
{
    SCON = 0x50;     // UART mode 1, 8-bit, REN enabled
    TMOD = 0x20;     // Timer1 mode 2 (auto-reload)
    TH1  = 0xFD;     // Baud rate 9600
    TR1  = 1;        // Start Timer1

    IE = 0x90;       // EA = 1, ES = 1 (enable serial interrupt)

    TI = 1;          // Enable printf

    while(1)
    {
        printf("Hello World\n");
        delay();
    }
}
```

OUTPUT:

a) **Displaying a Message:**



b) **During interruption:**



OBSERVATION:

Action	**Output in Serial Window**
Program running	Hello World printed repeatedly
Key pressed (e.g., A)	>>> INTERRUPT RECEIVED: A

Action	Output in Serial Window
Multiple keys pressed	Corresponding interrupt messages

INTERPRETATION:

- Continuous “Hello World” printing indicates **main program execution**
- Typing a key sets the **RI flag**
- Setting RI invokes the **serial interrupt**
- ISR executes immediately and prints the received character
- Interrupt output appears **between normal program outputs**
- This proves that the interrupt **pre-empts normal execution**

RESULT:

Thus, the serial interrupt using UART was successfully implemented and verified in Keil simulator. The interrupt was triggered upon keyboard input and serviced correctly using an Interrupt Service Routine.

Exp No: 17

UART INTERFACING USING INTERRUPT IN KEIL

AIM:

To receive serial data using interrupt.

APPARATUS REQUIRED:

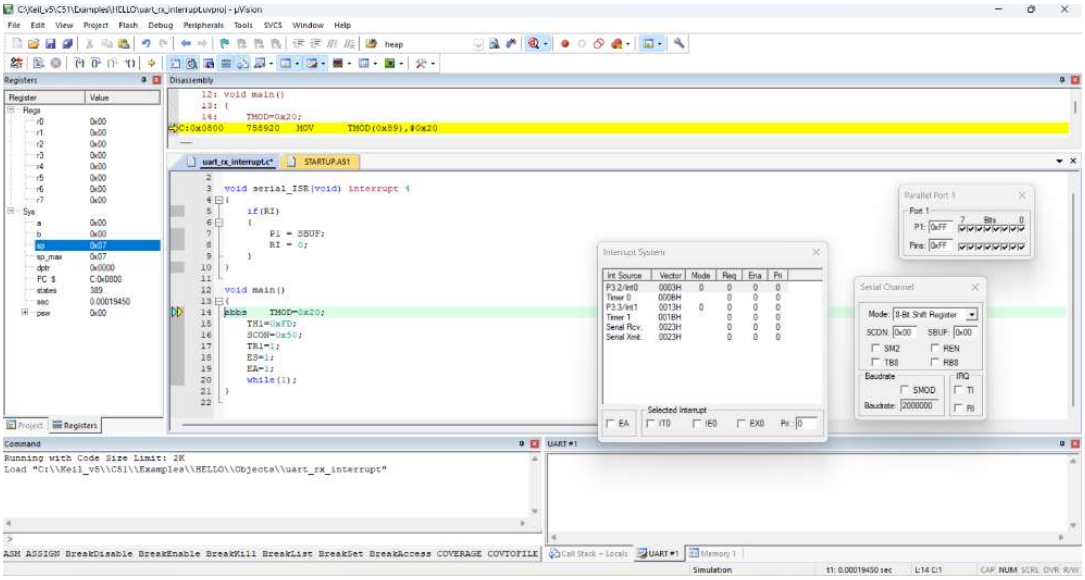
- PC with Windows OS
- Keil µVision IDE
- 8051 Simulator (Keil built-in)
- Serial Window (UART #1)

PROGRAM:

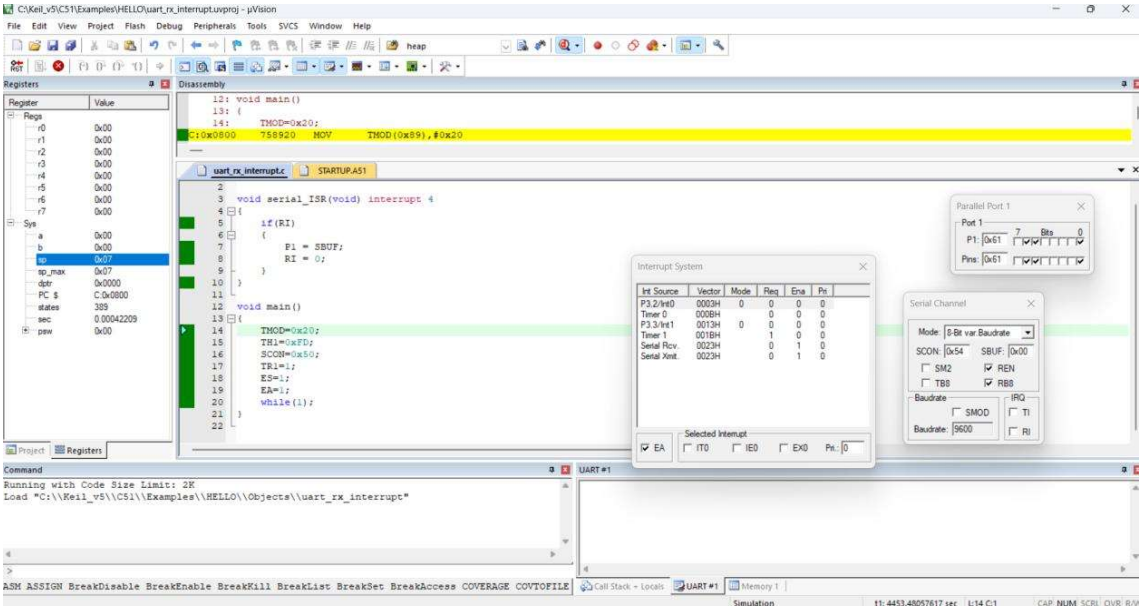
```
#include <reg51.h>

void serial_ISR(void) interrupt 4
{
   if(RI)
   {
      P1 = SBUF;
      RI = 0;
   }
}

void main()
{
   TMOD=0x20;
   TH1=0xFD;
   SCON=0x50;
   TR1=1;
   ES=1;
   EA=1;
   while(1);
}
```

OUTPUT:

Before interrupt:



After interrupt:



OBSERVATIONS:

Input: Character typed in UART window

Processing: Serial interrupt triggered

Output: Port 1 displays ASCII value of received character

Typed	ASCII	P1
A	41H	0x41
a	61H	0x61
1	31H	0x31

Received serial data appears on Port1.

RESULT:

Thus the serial data was received using Interrupt and the received data was verified.

Exp No: 18

GENERATION OF PWM USING TIMER INTERRUPT

AIM:

To generate PWM using Timer interrupt.

APPARATUS / SOFTWARE REQUIRED:

- PC with Windows OS
- Keil µVision IDE
- 8051 Simulator (Keil built-in)
- Serial Window (UART #1)

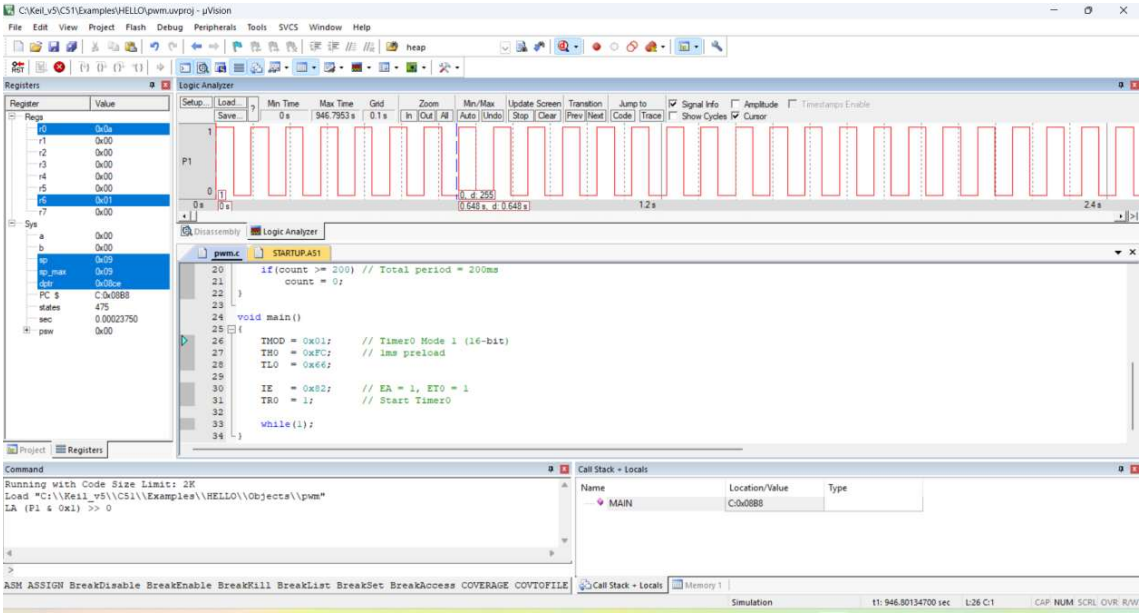
PROGRAM:

```
#include <reg51.h>

sbit PWM = P1^0;

unsigned char count = 0;
unsigned char duty = 100;   // Change this value (0–200)

void timer0_ISR(void) interrupt 1
{
   TH0 = 0xFC;      // Reload for 1ms delay
   TL0 = 0x66;

   count++;

   if(count < duty)
      PWM = 1;     // ON time
   else
      PWM = 0;     // OFF time

   if(count >= 200) // Total period = 200ms
```

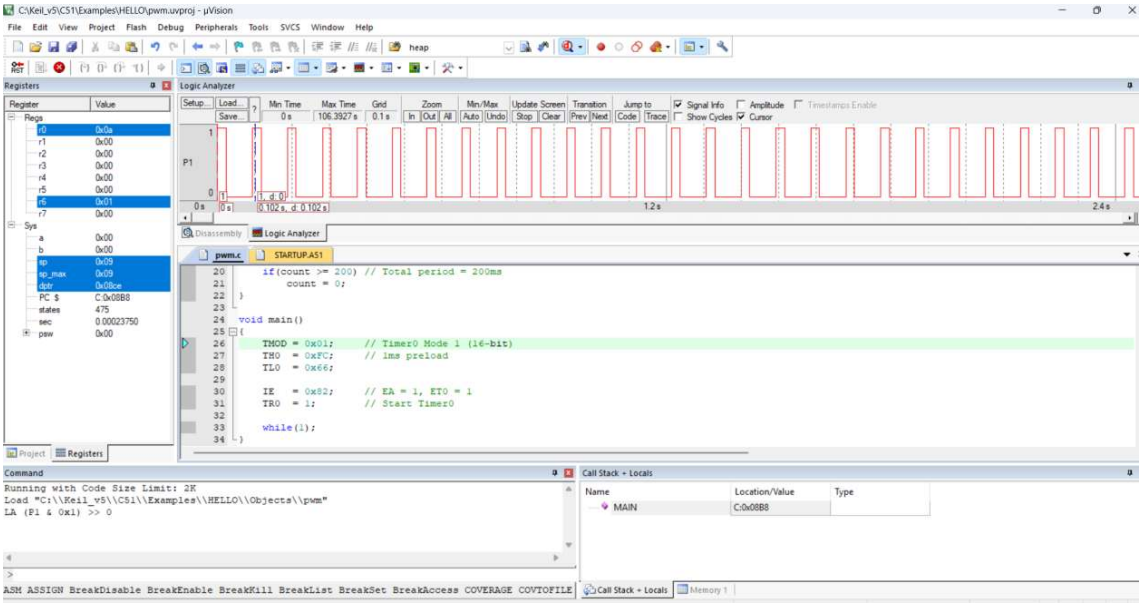
```
        count = 0;
}

void main()
{
   TMOD = 0x01;     // Timer0 Mode 1 (16-bit)
   TH0  = 0xFC;     // 1ms preload
   TL0  = 0x66;

   IE   = 0x82;     // EA = 1, ET0 = 1
   TR0  = 1;        // Start Timer0

   while(1);
}
```

OUTPUT:

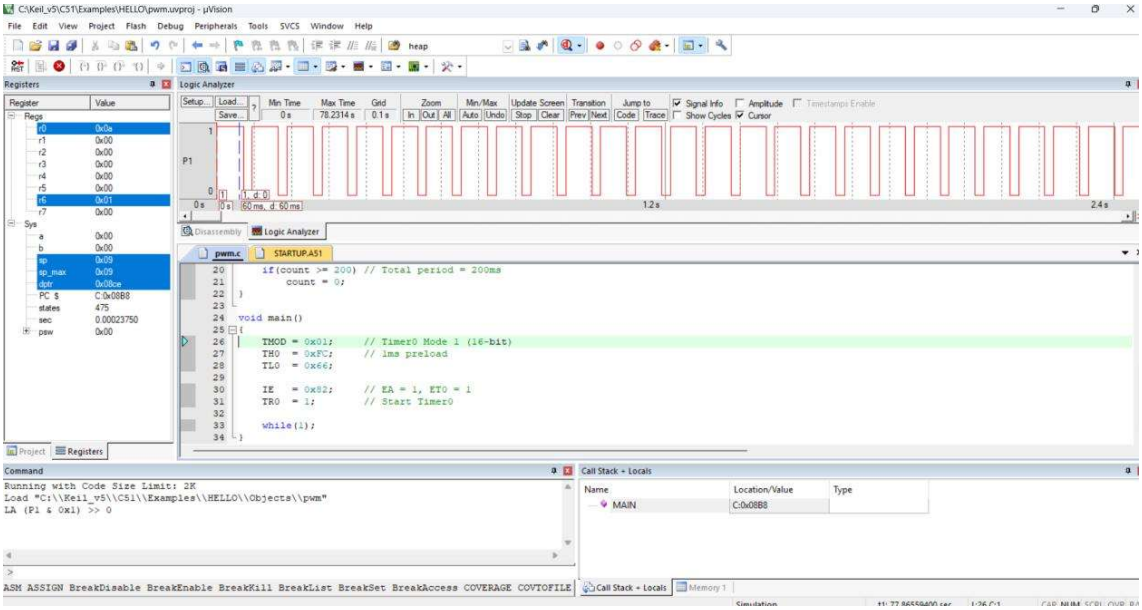
50% Duty cycle:



25% duty cycle:



75% duty cycle:



How to Change Duty Cycle:

Total period = 200 ms
Interrupt = 1 ms

Duty Value	Duty Cycle
50	25%
100	50%
150	75%
200	100%
0	0%

Formula:

Duty cycle (%) = (ON time / Total period) × 100

RESULT:

Thus the PWM was generated using Timer Interrupt for various duty cycle.

STUDY OF ARM PROCESSOR

AIM:

To study the architecture and working of an ARM processor.

COMPONENT REQUIRED:

- ARM LPC2148

THEORY:

ARM is a family of instruction set architectures for computer processors based on a reduced instruction set computing (RISC) architecture developed by British company ARM Holdings. A RISC-based computer design approach means ARM processors require significantly fewer transistors than typical processors in average computers. This approach reduces costs, heat and power use. These are desirable traits for light, portable, battery-powered devices—including smartphones, laptops, tablet and notepad computers), and other embedded systems. A simpler design facilitates more efficient multi-core CPUs and higher core counts at lower cost, providing higher processing power and improved energy efficiency for servers and supercomputers.

Features of LPC214x Series Controllers:

- 8 to 40 kB of on-chip static RAM and 32 to 512 kB of on-chip flash program memory.128 bit wide interface/accelerator enables high speed 60 MHz operation.
- In-System/In-Application Programming (ISP/IAP) via on-chip boot-loader software. Single flash sector or full chip erase in 400 ms and programming of 256 bytes in 1ms.
- Embedded ICE RT and Embedded Trace interfaces offer real-time debugging with the on-chip Real Monitor software and high speed tracing of instruction execution.
- USB 2.0 Full Speed compliant Device Controller with 2 kB of endpoint RAM. In addition, the LPC2146/8 provides 8 kB of on-chip RAM accessible to USB by DMA.
- One or two (LPC2141/2 vs. LPC2144/6/8) 10-bit A/D converters provide a total of 6/14analog inputs, with conversion times as low as 2.44 us per channel.
- Single 10-bit D/A converter provides variable analog output.
- Two 32-bit timers/external event counters (with four capture and four compare channels each), PWM unit (six outputs) and watchdog.
- Low power real-time clock with independent power and dedicated 32 kHz clock input.
- Multiple serial interfaces including two UARTs (16C550), two Fast I2C-bus (400 kbit/s), SPI and SSP with buffering and variable data length capabilities.
- Vectored interrupt controller with configurable priorities and vector addresses.
- Up to 45 of 5 V tolerant fast general purpose I/O pins in a tiny LQFP64 package.
- Up to nine edge or level sensitive external interrupt pins available.
- On-chip integrated oscillator operates with an external crystal in range from 1 MHz to30 MHz and with an external oscillator up to 50 MHz.
- Power saving modes include Idle and Power-down.
- Individual enable/disable of peripheral functions as well as peripheral clock scaling for additional power optimization.
- Processor wake-up from Power-down mode via external interrupt, USB, Brown-Out Detect (BOD) or Real-Time Clock (RTC).
- Single power supply chip with Power-On Reset (POR) and BOD circuits – CPU operating voltage range of 3.0 V to 3.6 V (3.3 V ± 10 %) with 5 V tolerant I/O pads.

LPC2148 needs the following hardware to work properly:

- Power Supply
- Crystal Oscillator
- Reset Circuit
- RTC crystal oscillator
- UART

Power Supply

LPC2148 works on 3.3 V power supply. LM 117 can be used for generating 3.3 V supply. However, basic peripherals like LCD, ULN 2003 (Motor Driver IC) etc. works on 5V. So AC mains supply is converted into 5V using below mentioned circuit and after that LM 117 is used to convert 5V into 3.3V.

Reset Circuit

Reset button is essential in a system to avoid programming pitfalls and sometimes to manually bring back the system to the initialization mode. MCP 130T is a special IC used for providing stable RESET signal to LPC 2148.

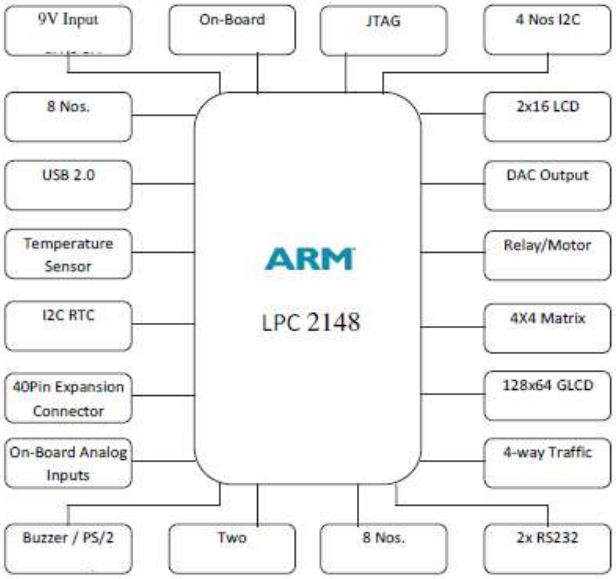


Figure - 3.1: ARM Processor

Flash Programming Utility

NXP Semiconductors produce a range of Microcontrollers that feature both on-chip. Flash memory and the ability to be reprogrammed using In-System Programming technology.

On-board Peripherals

• 8-Nos. of Point LED's (Digital Outputs)

• 8-Nos. of Digital Inputs (Slide Switch)

• 2 Lines X 16 Character LCD Display

• I2C Enabled 4 Digit Seven-Segment Display

• 128x64 Graphical LCD Display

• 4 X 4 Matrix keypad

• Stepper Motor Interface

• 2 Nos. Relay Interface

• Two UART for Serial Port Communication through PC

• Serial EEPROM

• On-chip Real Time Clock with Battery Backup

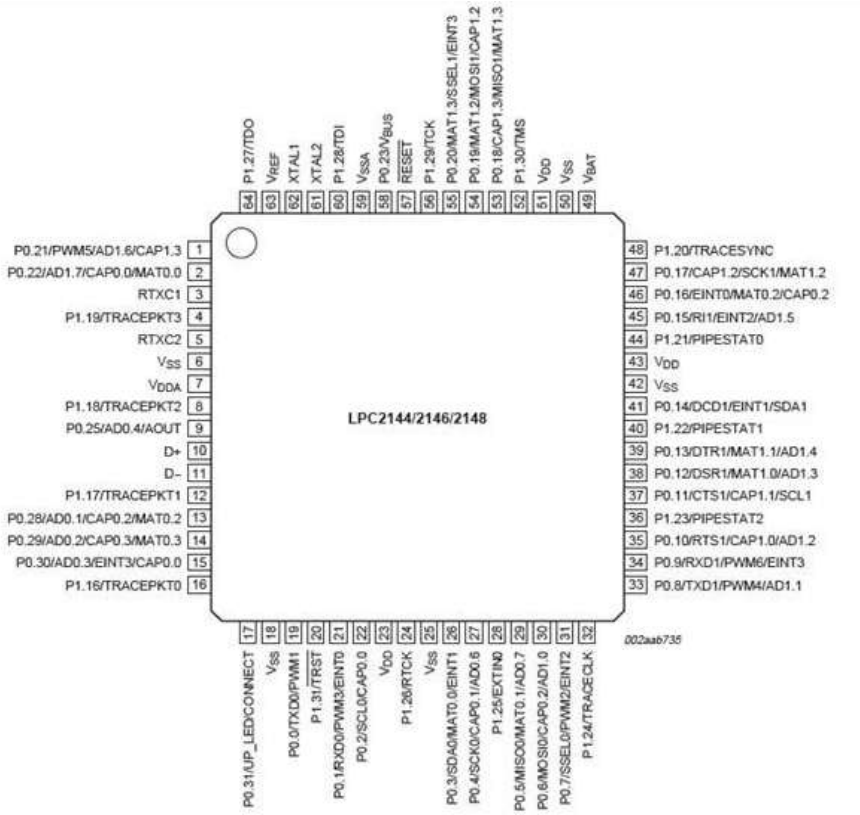
• PS/2 Keyboard Interface (Optional)

• Temperature Sensor

• Buzzer (Alarm Interface)

• Traffic Light Module (Optional)

Pin Configuration



Result:

The ARM processor has been studied successfully.

Exp No: 19

BLINKING LEDS USING SOFTWARE DELAY ROUTINE IN LPC2148 KIT

AIM:

To write and execute a C program to blink LEDs using software delay routine in LPC 2148 kit

APPARATUS REQUIRED:

Keil uVision5 Software

Philips Flah Programmer

LPC 2148 kit

PROGRAM:

```
#include "lpc214x.h"
void delay (unsigned int k);
void main(void)
{
IODIR0 = 0xFFFFFFFF; //Configure Port0 as output Port
PINSEL0 = 0;                    //Configure Port0 as General Purpose IO
while(1)
{
IOSET0 = 0x0000ff00;  //Set P0.15-P0.8 to '1'
delay(1000);          //1 sec Delay
IOCLR0 = 0x0000ff00;  //Set P0.15-P0.8 to '0'
delay(1000);          //1 Sec Delay
}
}
//Delay Program
//Input - delay value in milli seconds
void delay(unsigned int k)
{
        unsigned int i,j;
        for (j=0; j<k; j++)
```

```
                for(i = 0; i<=800; i++);
}
```

OUTPUT: LEDs P0.15-P0.8 are blinking

RESULT:

Thus the C program to blink LEDs using software delay routine was written and executed in LPC 2148 kit

Exp No: 20

PROGRAM TO READ THE SWITCH AND DISPLAY IN THE LEDS USING LPC2148 KIT

AIM:

To write and execute C program to read the switch and display in the LEDs using LPC2148 kit

APPARATUS REQUIRED:

Keil uVision5 Software

Philips Flah Programmer

LPC 2148 kit

PROGRAM:

```
#include "lpc214x.h"
int main(void)
{
  unsigned int sw_sts;
  IODIR0 = 0x0000ff00; //Configure Port0
        PINSEL0 = 0;                    //Configure Port0 as General Purose IO
while(1)
{
  sw_sts = IOPIN0;
  IOSET0 = 0x0000ff00;   //Set P0.15-P0.8 to '1'
  IOCLR0 = sw_sts >> 8;  //Set P0.15-P0.8 to '0'
}
}
```

OUTPUT: LEDs P0.15-P0.08 displayed the bits entered in the switches

RESULT:

Thus C program was written read the switch and display in the LEDs using LPC2148 kit

Exp No: 21

DISPLAY A NUMBER IN SEVEN SEGMENT LED IN LPC2148 KIT

AIM:

To write and execute C program to display a number in seven segment LED in LPC2148 kit

APPARATUS REQUIRED:

Keil uVision5 Software

Philips Flah Programmer

LPC 2148 kit

PROGRAM:

```
//SEVEN SEGMENT LED DISPLAY INTERFACE IN C
/* Program to Count 0-9 and Display it in 7 segment Display (MUX) DS4
 * Display Select DS3 ==> "P0.13"  Enable --> '0',  Disable --> '1'
 * Display Select DS4 ==> "P0.12"  Enable --> '0',  Disable --> '1'
 */

/* Segment Connection Display 1 & 2            Enable --> '1',  Disable --> '0'
 *-------------------------------------------------------------------
 * MSB                                        LSB
 *   Dp    G     F     E     D     C     B     A
 * P0.23  P0.22  P0.21  P0.20  P0.19  P0.18  P0.17  P0.16
 *   0     0     0     0     0     1     1     0   --> 6 => '1'
 *-------------------------------------------------------------------*/

#include <LPC214X.H>
#define DS3   1<<13             // P0.13
#define DS4   1<<12             // P0.12
#define SEG_CODE 0xFF<<16     // Segment Data from P0.16 to P0.23
unsigned char const seg_dat[]={0x3F, 0x6, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x7, 0x7F,
0x67};
```

```
void delayms(int n)
{
int i,j;
for(i=0;i<n;i++)
 {for(j=0;j<5035;j++)   //5035 for 60Mhz ** 1007 for 12Mhz
    {;}
 }
}

int main (void)
{
  unsigned char count;

PINSEL0 = 0; // Configure Port0 as General Purpose IO => P0.0  to P0.15
PINSEL1 = 0; // Configure Port0 as General Purpose IO => P0.16 to P0.31
IODIR0 = SEG_CODE | DS3 | DS4; //Configure Segment data & Select signal as output
IOSET0 = SEG_CODE | DS3 ;  //Disable DS3 display
IOCLR0 = DS4;  //Enable DS4 Display
count  = 0; //Initialize Count

//Display Count value
IOCLR0 = SEG_CODE;
IOSET0 = seg_dat[count]<<16;

while(1)
 {
          delayms(1000); //1 sec delay
   count++;              //Increment count
   if(count>9) count=0;  //Limit 0-9
```

```
    //Display Count value
    IOCLR0 = SEG_CODE;
    IOSET0 = seg_dat[count]<<16;


   }
}
```

OUTPUT: 7-Segment display counting from 0 to 9

RESULT:

Thus C program, was written and executed to display a number in seven segment LED in LPC2148 kit